Skill – Week7

Session 13:

1. To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

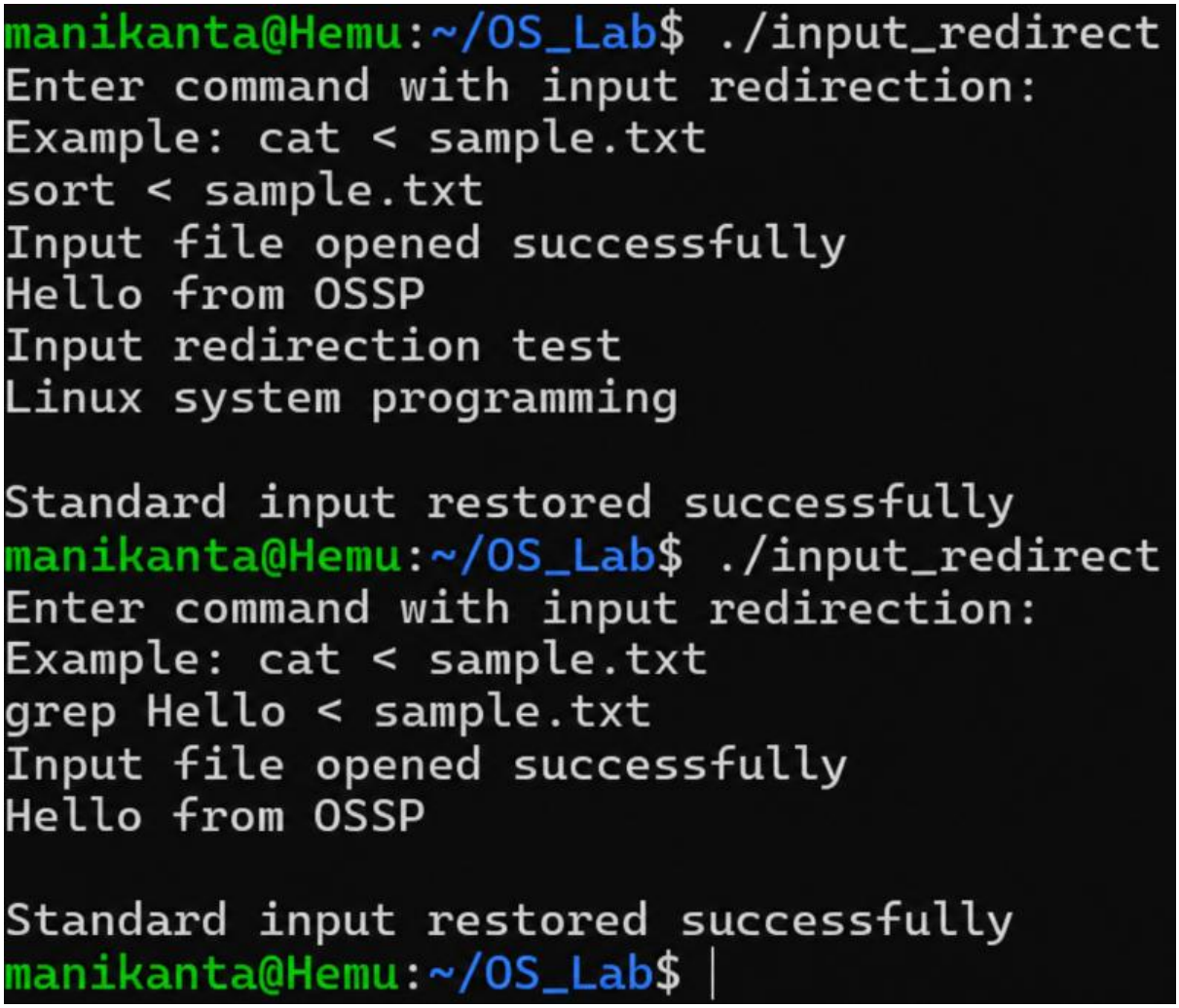
#define MAX_INPUT 200
#define MAX_ARGS 20

int main() {
    char input[MAX_INPUT];
    char *args[MAX_ARGS];
    int fd;
    int saved_stdin;
    int count = 0;
    printf("Enter command with input redirection:\n");
    printf("Example: cat < sample.txt\n");
    fgets(input, MAX_INPUT, stdin);
    input[strcspn(input, "\n")] = '\0';
    /* Save original stdin */
    saved_stdin = dup(STDIN_FILENO);
    if (saved_stdin == -1) {
        perror("dup");
        return 1;
    }
    /* Parse command arguments */
    char *token = strtok(input, " ");
    while (token != NULL && count < MAX_ARGS - 1) {
        if (strcmp(token, "<") == 0) {
            /* Get input filename */
            token = strtok(NULL, " ");
            if (token == NULL) {
                printf("Error: Missing input filename\n");
                close(saved_stdin);
                return 1;
            }
```

```
manikanta@Hemu:~/OS_Lab$ echo "Hello from OSSP" > sample.txt
manikanta@Hemu:~/OS_Lab$ echo "Input redirection test" >> sample.txt
manikanta@Hemu:~/OS_Lab$ echo "Linux system programming" >> sample.txt
manikanta@Hemu:~/OS_Lab$ ls sample.txt
sample.txt
manikanta@Hemu:~/OS_Lab$ cat sample.txt
Hello from OSSP
Input redirection test
Linux system programming
manikanta@Hemu:~/OS_Lab$ nano input_redirect.c
manikanta@Hemu:~/OS_Lab$ gcc input_redirectc.-o input_redirect
manikanta@Hemu:~/OS_Lab$ ./input_redirect
Enter command with input redirection:
Example: cat < sample.txt
cat < sample.txt
Input file opened successfully
Hello from OSSP
Input redirection test
Linux system programming

Standard input restored successfully
manikanta@Hemu:~/OS_Lab$ ./input_redirect
Enter command with input redirection:
Example: cat < sample.txt
wc -l < sample.txt
Input file opened successfully
3

Standard input restored successfully
manikanta@Hemu:~/OS_Lab$ ./input_redirect
Enter command with input redirection:
Example: cat < sample.txt
wc -w < sample.txt
Input file opened successfully
9
```



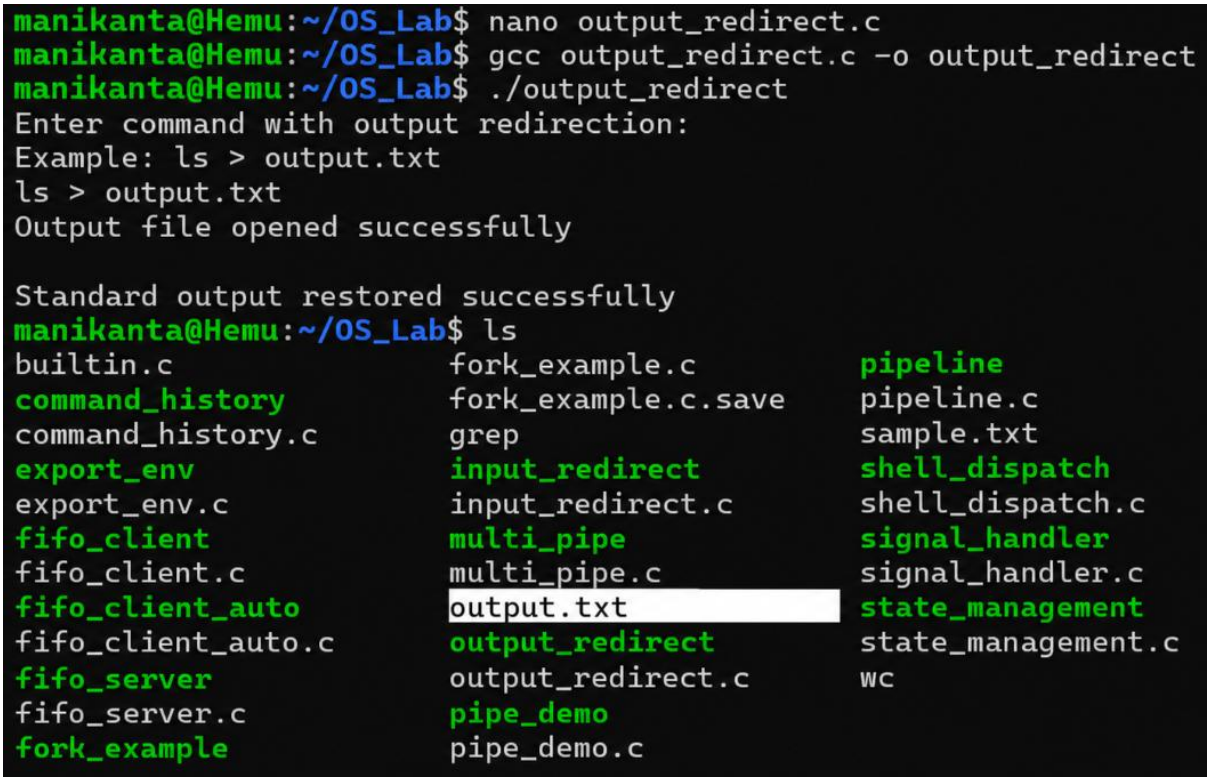
Report: The program successfully parses input redirection, opens sample.txt, redirects standard input, executes the command, and restores the original standard input. File input is tested using cat, wc, sort, and grep.

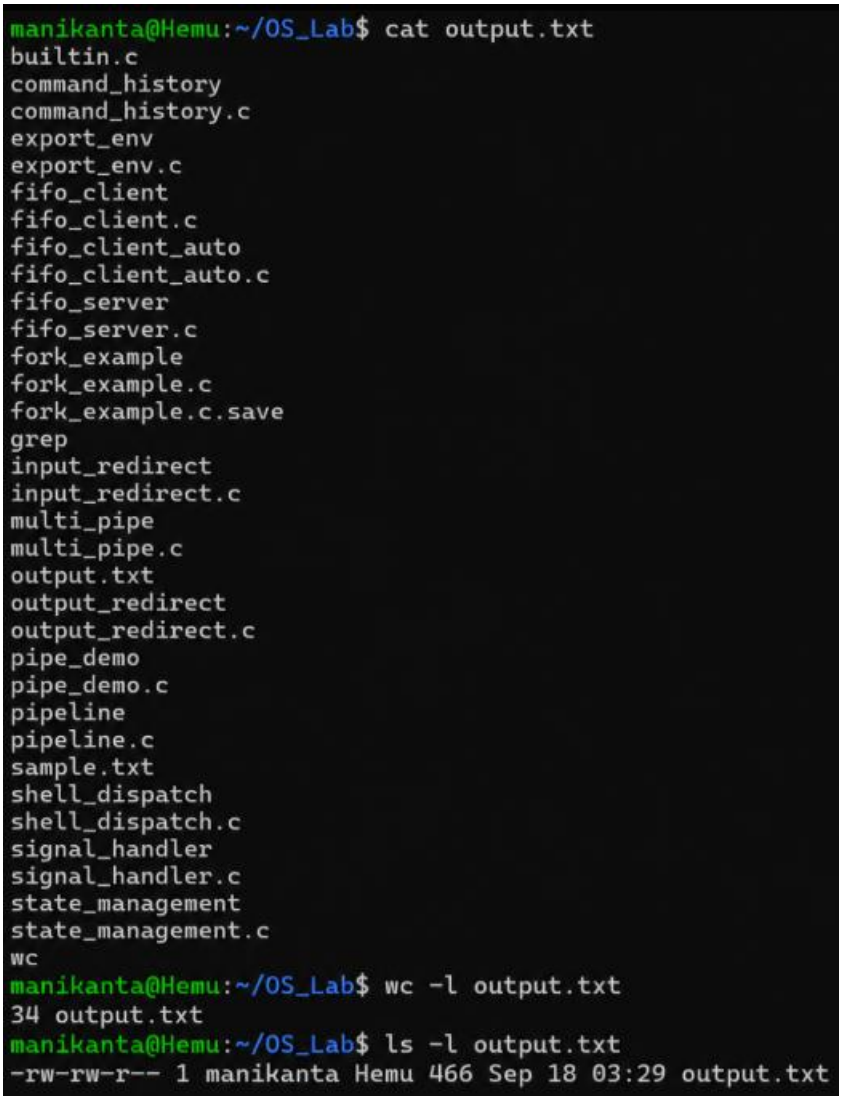
2. To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_INPUT 200
#define MAX_ARGS 20

int main() {
    char input[MAX_INPUT];
    char *args[MAX_ARGS];
    int fd;
    int saved_stdout;
    int count = 0;
    printf("Enter command with output redirection:\n");
    printf("Example: ls > output.txt\n");
    fgets(input, MAX_INPUT, stdin);
    input[strcspn(input, "\n")] = '\0';
    /* Save original stdout */
    saved_stdout = dup(STDOUT_FILENO);
    if (saved_stdout == -1) {
        perror("dup");
        return 1;
    }
    /* Parse command arguments */
    char *token = strtok(input, " ");
    while (token != NULL && count < MAX_ARGS - 1) {
        if (strcmp(token, ">") == 0) {
            /* Get output filename */
            token = strtok(NULL, " ");
            if (token == NULL) {
                printf("Error: Missing output filename\n");
                close(saved_stdout);
                return 1;
            }
```

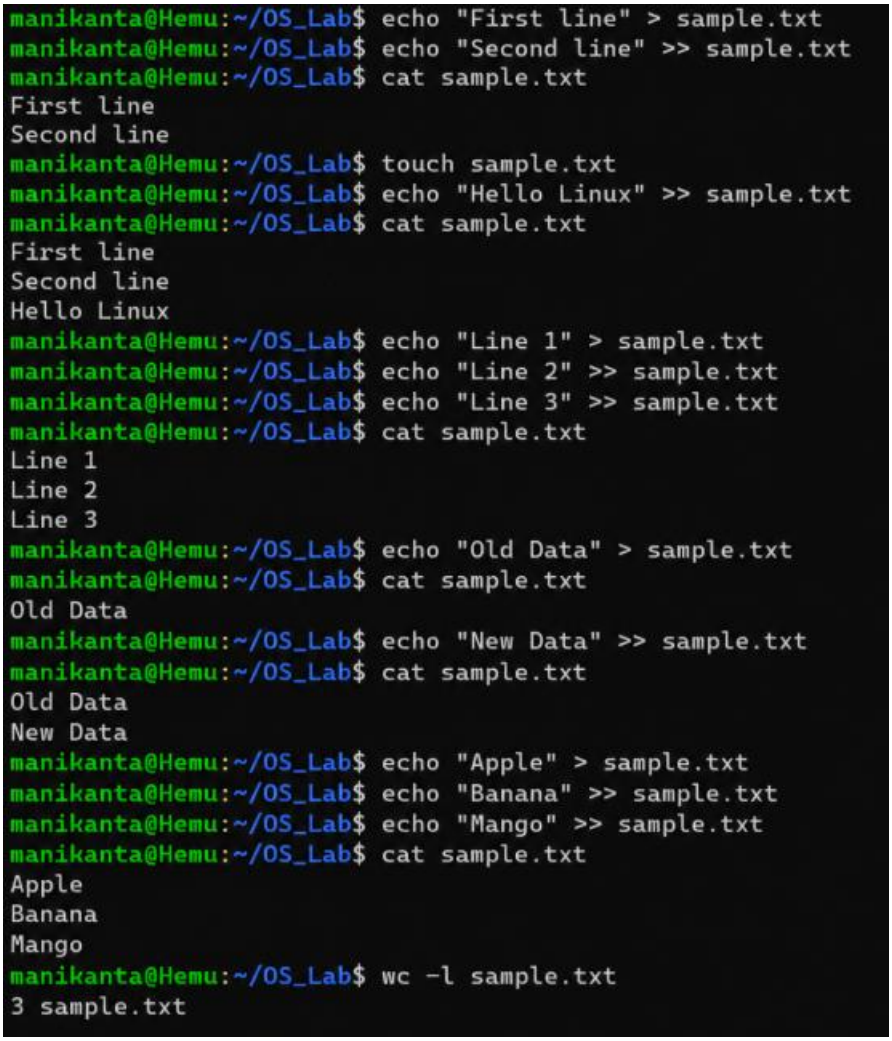

manikanta@Hemu:~/OS_Lab$ nano output_redirect.c
manikanta@Hemu:~/OS_Lab$ gcc output_redirect.c -o output_redirect
manikanta@Hemu:~/OS_Lab$ ./output_redirect
Enter command with output redirection:
Example: ls > output.txt
ls > output.txt
Output file opened successfully
Standard output restored successfully
manikanta@Hemu:~/OS_Lab$ ls
builtin.c
command_history
command_history.c
export_env
export_env.c
fifo_client
fifo_client.c
fifo_client_auto
fifo_client_auto.c
fifo_server
fifo_server.c
fork_example
fork_example.c
fork_example.c.save
grep
input_redirect
input_redirect.c
multi_pipe
multi_pipe.c
output.txt
output_redirect
output_redirect.c
pipe_demo
pipe_demo.c
pipeline
pipeline.c
sample.txt
shell_dispatch
shell_dispatch.c
signal_handler
signal_handler.c
state_management
state_management.c
wc



Report: The program parses output redirection using >, creates an output file using open(), redirects standard output using dup2(), handles file permission errors, restores the original output stream, and verifies the redirected data using cat and wc.

Session 14:

1. To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios



Report: The append operator >> adds new data to the end of a file while preserving existing content. The commands and C program demonstrate file appending, output redirection, and validation.

2. To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases

```c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd;

    fd = open("error.txt",
              O_WRONLY | O_CREAT | O_APPEND, 0644);

    if (fd == -1) {
        perror("Error opening file");
        return 1;
    }

    dup2(fd, STDERR_FILENO);

    close(fd);

    fprintf(stderr, "Error: File not found\n");
    fprintf(stderr, "Error: Invalid command\n");

    return 0;
}
```

```
manikanta@Hemu:~/OS_Lab$ nano stderr_redirect.c
manikanta@Hemu:~/OS_Lab$ gcc stderr_redirect.c -o stderr_redirect
manikanta@Hemu:~/OS_Lab$ ./stderr_redirect
manikanta@Hemu:~/OS_Lab$ cat error.txt
First error
ls: cannot access '/wrong1': No such file or directory
ls: cannot access '/wrong2': No such file or directory
Error: File not found
Error: Invalid command
```

```
manikanta@Hemu:~/OS_Lab$ cat error.txt
cat: /wrong_file: No such file or directory
manikanta@Hemu:~/OS_Lab$ echo "Old error" > error.txt
manikanta@Hemu:~/OS_Lab$ ls /missing_folder 2>> error.txt
manikanta@Hemu:~/OS_Lab$ cat error.txt
Old error
ls: cannot access '/missing_folder': No such file or directory
manikanta@Hemu:~/OS_Lab$ ls /invalid_path 2> error.txt

if [ $? -ne 0 ]; then
    echo "Command failed"
else
    echo "Command succeeded"
fi
Command failed
manikanta@Hemu:~/OS_Lab$ ls /not_found 2> error.txt
manikanta@Hemu:~/OS_Lab$ cat error.txt
ls: cannot access '/not_found': No such file or directory
manikanta@Hemu:~/OS_Lab$ cat missing.txt 2> error.txt
manikanta@Hemu:~/OS_Lab$ cat error.txt
cat: missing.txt: No such file or directory
manikanta@Hemu:~/OS_Lab$ cat /root/secret.txt 2> error.txt
manikanta@Hemu:~/OS_Lab$ cat error.txt
cat: /root/secret.txt: Permission denied
manikanta@Hemu:~/OS_Lab$ echo "First error" > error.txt
manikanta@Hemu:~/OS_Lab$ ls /wrong1 2>> error.txt
manikanta@Hemu:~/OS_Lab$ ls /wrong2 2>> error.txt
manikanta@Hemu:~/OS_Lab$ cat error.txt
First error
ls: cannot access '/wrong1': No such file or directory
ls: cannot access '/wrong2': No such file or directory
```

Report: stderr redirection is used to capture error messages into a file using 2> and 2>>. The commands and C program demonstrate error handling, preserving previous data, and verifying error output.